

INNOVATION BRIEF

LegalEase.

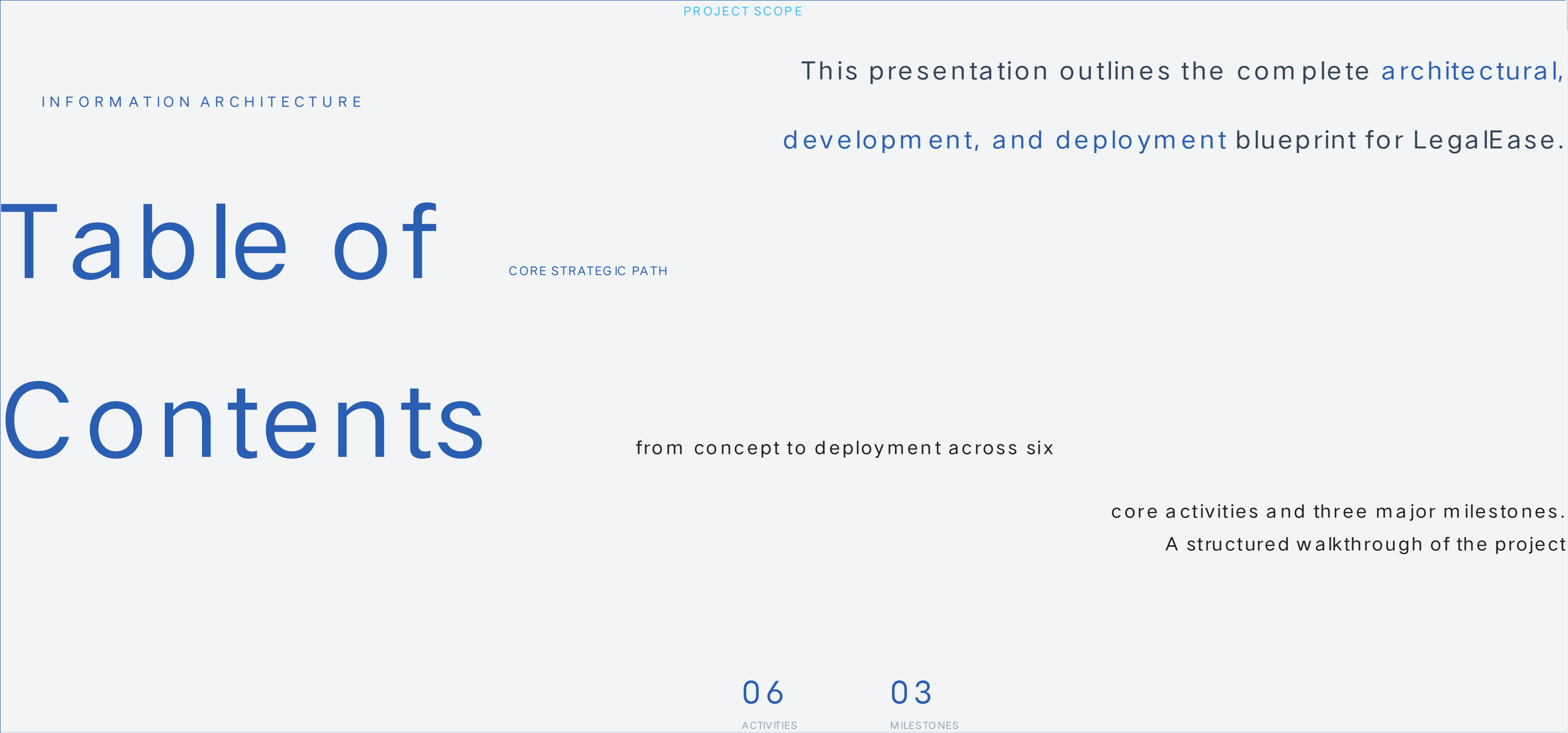
AI-Powered Legal Document Generator

An overview of LegalEase, a platform leveraging state-of-the-art Generative AI to simplify the creation of customizable, accurate, and editable legal documents.

CORE MISSION

Bridging the Gap Between Accessibility and Legal Professionalism

SYSTEM STATUS: ACTIVE
GENAI FRAMEWORK V.2.4



Legal Technologist v1.0

Project Blueprint

AGENDA

06

ACTIVITIES

03

MILESTONES

CHAPTER 01

Project Overview & Scenarios

Demonstrating how LegalEase solves legal document generation challenges for startups, freelancers, and landlords.

SYSTEM STATUS

PHASE_01 // ARCHITECTURAL_INIT

SECTION 01-A

Real-World Application Scenarios

Three practical use cases showing how different target users interact with LegalEase to generate formal documents.



Startup Founder

NEED

Employment Contract for a new hire.

ACTION

Enters details about roles, responsibilities, compensation, and confidentiality.

OUTCOME

Generates a comprehensive agreement, customizes wording, adds company logo,



Freelancer

NEED

Non-Disclosure Agreement (NDA) for a potential client.

ACTION

Specifies involved parties, scope of confidentiality, and effective date.

OUTCOME

Instantly receives a structured and legally



Landlord

NEED

Residential Lease Agreement.

ACTION

Inputs property address, tenant details, and lease terms.

OUTCOME

Creates a formatted document with editable sections downloadable in .DOCX

- LegalEase addresses distinct user needs by providing tailored legal outputs.

CHAPTER 02

Technical Pre-requisites & Architecture

Establishing a robust foundation using FastAPI,
Streamlit, and Google Gemini 1.5 Pro.

Technical Pre-requisites & Environment Setup

Core Framework Knowledge Required



FastAPI Framework

For building high-performance, production-ready backend APIs.



Google Gemini API

Familiarity with Google Generative AI SDK for advanced text generation.



Streamlit

For rapid frontend UI development and interactive components.



Python Programming

High proficiency required for core logic and integration.

"The foundational knowledge and specific Python libraries required to build and execute the LegalEase platform."

PROJECT BASIS

A standard Python 3.10+ environment equipped with specialized AI and document formatting SDKs forms the project basis.

Environment Installation Command

```
zsh — setup.sh

$ pip install fastapi uvicorn streamlit python-docx
fpdf Pillow requests google-generativeai python-dotenv
```

System Architecture & Component Definition

LegalEase utilizes a decoupled architecture separating the Streamlit UI from the FastAPI backend for optimal scalability.

- Frontend (Streamlit)

User interaction, document input fields, responsive layouts, dynamic previews, and file downloads.

- Backend (FastAPI)

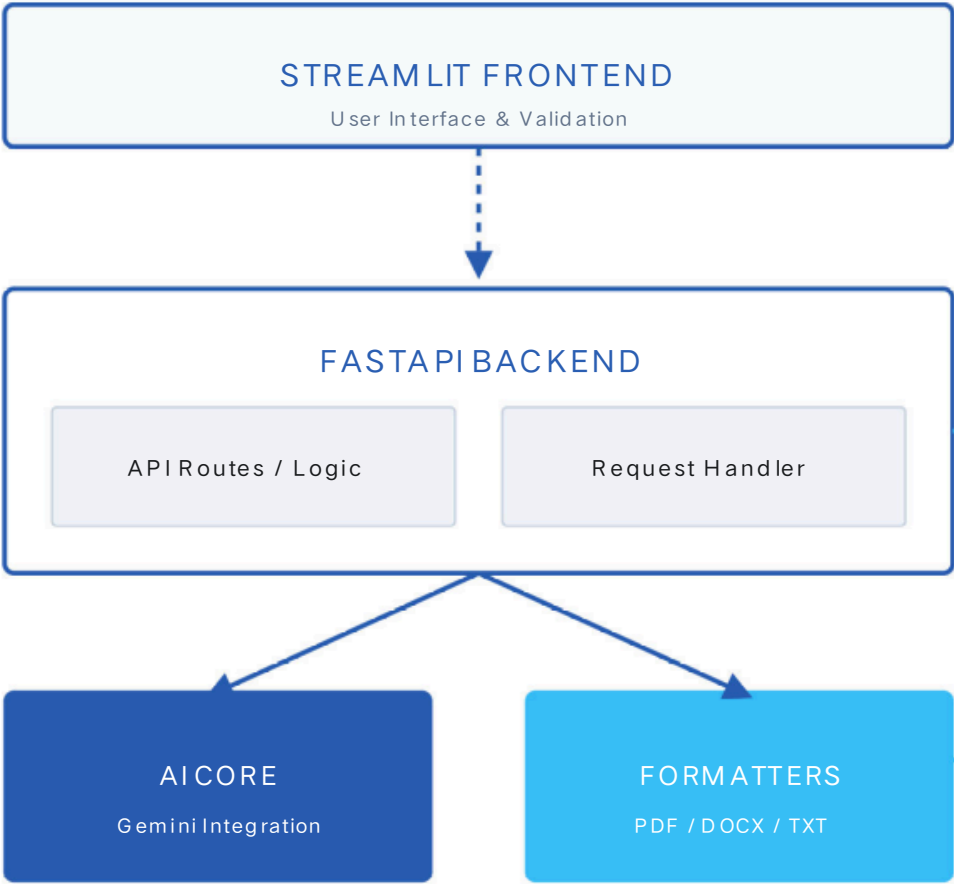
API routing (main.py, routes.py), processes incoming POST requests, and communicates with the AI core.

- AI Core (Gemini)

Encapsulated inside gemini_generation.py to handle prompt engineering and model execution.

- Formatting Modules

ARCHITECTURE DIAGRAM V1.0.4



CHAPTER 03

Milestone 1: Model Selection & Environment

"Selecting the optimal generative model capable of handling complex legal jargon and multi-part structured prompts."

SYSTEM.LATENT
0.0031_PRO_VERSION

STATUS
ENVIRONMENT_READY

Model Selection: Why Gemini 1.5 Pro?

Gemini 1.5 Pro provides the necessary reasoning depth and context length required for professional legal drafting.



Milestone 2: Core Functionalities Development

Translating user inputs into structured legal documents through programmatic API management.

Backend Generation Logic & Prompt Engineering

Technical code structure and prompt engineering strategies used to interface the backend with the Gemini API.

Strict Pydantic schemas ensure valid data transmission from the frontend to the AI generation core.

FastAPI Endpoint Mapping

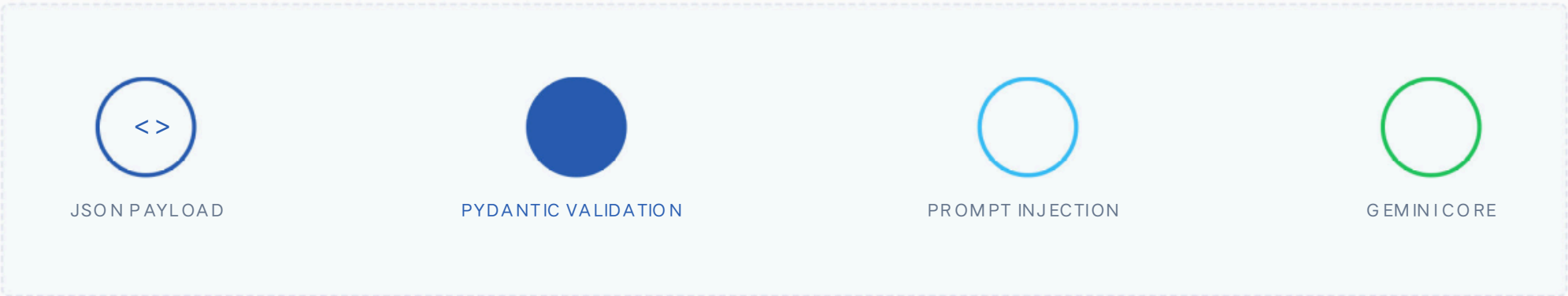
- Exposes a **POST /generate** endpoint for JSON payloads.
- Structured via a Pydantic model: **DocumentRequest**.
- Fields processed: document_type, parties, terms, dates.

0 The constructed prompt is sent via `model.generate_content(prompt)`, returning raw structured text back to the route handler.

GEMINI_GENERATOR.PY

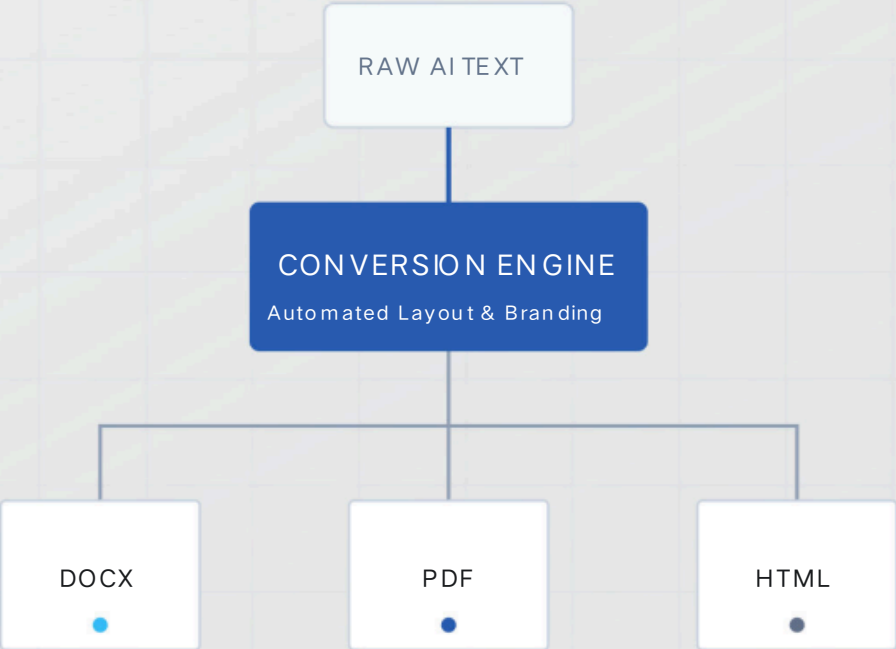
PROMPT CONSTRUCTION LOGIC

```
prompt = f"Generate a comprehensive legal document titled '{document_type}'\n" \
        f"Involved parties: {parties}\n" \
        f"Effective Date: {dates}\n" \
        f"Terms and conditions: {terms}\n" \
        f"Ensure formal legal structure with multiple sections and legal clauses."
```



Document Output Formatting & Exporters

LegalEase bridges the gap between raw text and professional presentation through automated layout engines.



MULTI-FORMAT CONVERSION ENGINE

W `format_docx(text, doc_type)`

- Converts raw text into Microsoft Word format.
- Automatically embeds custom company logos and branding elements.
- Sets typography to professional standards (Times New Roman).
- Formats defined terms into structured tables and appends legal footers.

`format_pdf(text, doc_type)`

- Utilizes the fpdf library for high-precision compilation.
- Ensures clean, branded PDF documents with unalterable formatting.
- Features consistent dynamic headers and automated page numbering.

</> `format_html_preview(text)`

- Renders a styled preview directly inside the Streamlit user interface.

Frontend UI & Interactive Session State

Detailed breakdown of the Streamlit frontend implementation, showing input collection and stateful editing features.

User Interface Components

Input Management

Employs `st.text_input` for concise variables and `st.text_area` for multi-line content.

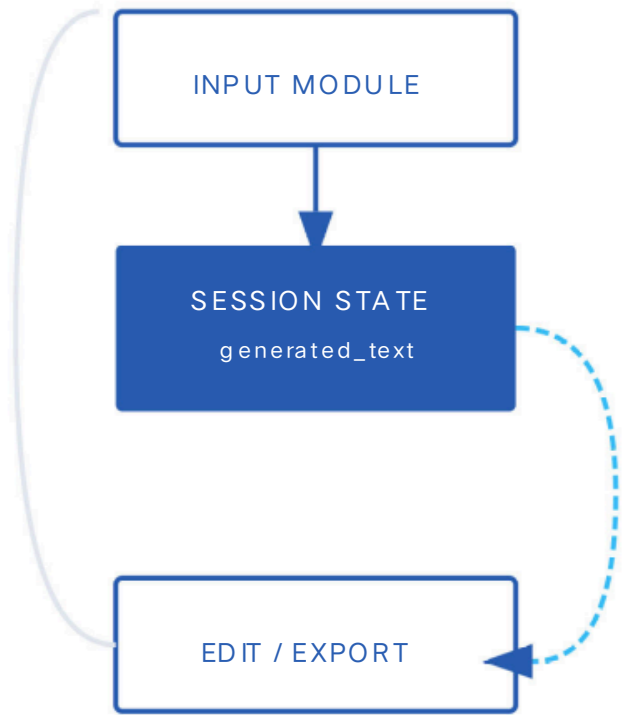
Stateful Editing

Direct text modification enabled when `show_edit` condition is met.

CORE PHILOSOPHY

The frontend provides an **intuitive, interactive workspace** allowing immediate text modification and multi-format export.

SESSION STATE LOGIC LOOP



Real-time synchronization of app.py

Action Triggers

TXT DOWNLOAD

`st.download_button(...)`

DOCX DOWNLOAD

W

`format_docx(data)`

PDF DOWNLOAD

`format_pdf(data)`

* Interactive text area rendered at 300px height upon state trigger.

TRANSITIONING THE APPLICATION FROM A LOCAL
DEVELOPMENT ENVIRONMENT TO CLOUD
INFRASTRUCTURE.

05 Deployment & Project Conclusion

Deployment Strategies & Final Testing

SLIDE OVERVIEW

The final steps required to package, deploy, and verify the LegalEase ecosystem in a production cloud environment.

CORE PRINCIPLE

Decoupled deployment allows independent scaling of the Streamlit frontend and the FastAPI backend.

Cloud Hosting Infrastructure Options

Frontend Hosting

Streamlit Community Cloud, Render, or Railway for rapid UI deployment.

Backend Hosting

Fly.io, Railway, or dedicated Virtual Private Servers (VPS) to manage sustained FastAPI request traffic.

Production Preparation

Consolidate production-ready dependencies within a verified `requirements.txt` file.

Configure operational runtime parameters using a `Procfile` or standard `Dockerfile` configurations.

Activity 6: Testing & QA

Conduct rigorous end-to-end functionality testing across diverse input combinations.

Validate Gemini API integration stability under varying network conditions and edge-case inputs.

Benefits of LegalEase

How the platform improves the legal document creation workflow

- Saves time by generating structured legal documents from user-provided details.
- Supports customizable and editable outputs before the final document is exported.
- Provides professional formatting in DOCX, PDF, and HTML preview formats.
- Connects a Streamlit frontend, FastAPI backend, and Gemini AI generation core.
- Supports practical scenarios such as employment contracts, NDAs, and lease agreements.

Future Enhancements & Safety

Potential improvements for a more reliable production-ready system

- Expand the library of legal document templates and supported scenarios.
- Add stronger validation for user inputs and generated clauses.
- Improve testing using diverse inputs, edge cases, and API/network failure conditions.
- Strengthen privacy and security controls for sensitive legal information and generated files.
- Provide a clear review step so important documents can be checked by a qualified legal professional.

FINAL STATEMENT

THANK YOU

SmartBridge — Let's Bridge the Gap.

Empowering users with accessible, professional legal document workflows through the fusion of technical precision and institutional authority.